DevSys VM — Incus Golden-Image Runbook

Identical, version-pinned Ubuntu dev VMs on a bare-metal Incus host — build one golden image, clone per developer in seconds, isolated over Tailscale.

The system: three files on a bare-metal Ubuntu server running Incus — `devsys-vm.sh`, `newvm.sh` and `template.yaml` in the `devsys-incus` folder, published in the gist as `devsys-v3.sh`, `newvm-v3.sh` and `template-v3.yaml`. You install the pinned toolchain once into a base VM, publish it as the image `devsys-base`, and every developer VM is a ~5-second clone. No Homebrew (mise installed directly, system-wide, as root); tools chosen by group, every version pinned. Each clone auto-joins Tailscale tagged `tag:devsys` + `tag:<dev>`; the admin reaches everything, each dev reaches only their own VM.

Incus or OrbStack

The same installer also runs on a Mac with OrbStack; only the host scripts differ. That setup lives in the `devsys-orbstack` folder with its own README.

Concern	Incus (this runbook)	OrbStack (`devsys-orbstack`)
Host	Ubuntu server + Incus	Mac (Apple Silicon) + OrbStack
Guest type	full virtual machine	OrbStack machine (shared kernel)
Golden image	published image `devsys-base`	the stopped machine `devsys-base`
Per-VM config	cloud-init user-data	`newvm.sh` runs `devsys install none`
OAuth creds	mode-600 file on host	env, mode-600 file, or macOS Keychain
Host filesystem	never mounted into guests	hidden by `--isolated`
Installer	`devsys-vm.sh` 3.3.0	the same file

Phase 1 — Build the golden image (one-time)

1.1 Create the base VM

Incus Web UI → Create instance. Image `ubuntu/24.04/cloud` (the cloud variant — it has cloud-init, which the clones need), type Virtual machine, name `devsys-base`, ~2 vCPU / 4 GiB / 40 GiB disk. Get a root shell via the Terminal tab (the Console tab wants a password the cloud image has none of).

1.2 Install the toolchain — tools only, no user, no Tailscale

```
GIST=https://gist.githubusercontent.com/secureedge26/574346b153a83b45062460d5433d8a60
curl -fsSL $GIST/raw/devsys-v3.sh | DEVSYS_UNATTENDED=1 bash
```

Omitting `DEVSYS_USER` and `DEVSYS_TS_AUTHKEY` installs the pinned tools and nothing VM-specific. Verify: `exec bash && node --version` → `v26.5.0`, and `devsys status` lists no tool as missing. (Add `DEVSYS_SHARED_CLAUDE=1` here if you want shared-Claude baked in — see the multi-user section.)

Needs installer 3.1.7 or later (this folder has 3.3.0). With 3.1.6 a default install can skip the `node`, `python`, `cloud` and `ai` groups and still print “Done”: a dpkg prompt about `/etc/zsh/zshenv` swallows the rest of the group list. This was seen on OrbStack; the code path is the same here. Publish `devsys-incus/devsys-vm.sh` to the gist as `devsys-v3.sh` before building a base.

1.3 Generalize + publish

In the base Terminal, strip the per-machine identity (mandatory — or every clone collides), then power off:

```
systemctl stop tailscaled 2>/dev/null || true
rm -f /var/lib/tailscale/tailscaled.state
: > /etc/machine-id
rm -f /var/lib/dbus/machine-id && ln -sf /etc/machine-id /var/lib/dbus/machine-id
rm -f /etc/ssh/ssh_host_*
cloud-init clean --logs
poweroff
```

Then, with the VM Stopped: Web UI → `devsys-base` → Create Image → Alias `devsys-base`. (`./build-base.sh` on the host does this whole step.)

Phase 2 — Tailscale setup (one-time, tailnet-level)

All in the Tailscale admin console, independent of the host — the OrbStack setup uses the same tags and rules.

2.1 Tags (Definitions → Tags)

Tag	Owner
`tag:devsys`	`autogroup:admin, tag:devsys`
`tag:soheb`	`tag:devsys`
(one per future dev)	`tag:devsys`

`tag:devsys` must own itself — multi-tag keys require every tag, including tag:devsys, to pass the ownership check.

2.2 Tailscale SSH rules (Access controls → Tailscale SSH)

#	Sources	Destination	Users
1	`autogroup:member`	`autogroup:self`	`autogroup:nonroot`, `root`
2	`autogroup:admin`	`tag:devsys`	`root`
3	`soheb@example.com`	`tag:soheb`	`soheb`
+1 per dev	`<dev>@example.com`	`tag:<dev>`	`<dev>`

Rule 2 = your admin root reach across the fleet; rules 3+ = each dev into only their own VM. Never use `tag:devsys` as a source — it would let every VM reach every other VM.

2.3 OAuth client + host creds

Settings → OAuth clients → scope Auth Keys: Write, Tags = `tag:devsys` + every person-tag. Store on the host in a mode-600 file (for the `newvm.sh` automation):

```
printf 'DEVSYS_TS_CLIENT_ID=%s\nDEVSYS_TS_CLIENT_SECRET=%s\n' ID SECRET \
  > ~/.config/devsys/ts-oauth.env && chmod 600 ~/.config/devsys/ts-oauth.env
```

Phase 3 — Clone a developer VM (per VM)

Two paths, same result. Both need a generated auth key — see the rules table below.

Path A — Web UI (no host shell)

1. Tailscale → Keys → Generate auth key: Reusable off, Ephemeral OFF, Pre-approved on, Tags `tag:devsys` + `tag:<dev>` → copy the `tskey-auth-…`.

2. Web UI → Local images → `devsys-base` → launch → name it, VM, and set Cloud init → User data (override pencil):

```
#cloud-config
hostname: sohebvm
runcmd:
  - >
    DEVSYS_UNATTENDED=1 DEVSYS_USER=soheb
    DEVSYS_TS_HOSTNAME=sohebvm DEVSYS_TS_AUTHKEY=tskey-auth-XXXX
    /usr/local/bin/devsys install none
```

3. Create and start — cloud-init sets hostname, creates the dev user, joins Tailscale (seconds; toolchain already baked).

Path B — one command on the host

```
./newvm.sh sohebvm soheb
```

Mints a dual-tagged, non-ephemeral `tskey-auth-…` from the OAuth client, launches the clone, injects the cloud-init. No manual key, no secret on the VM. `--login-link` for dev-owned nodes, `--project <dev>` for per-dev Incus projects.

Auth-key rules — the four things that bite (all avoided by Path B)

Rule	Why
Use a generated auth key (`tskey-auth-`), never the OAuth client secret (`tskey-client-`).	The client secret is for scripts to MINT keys; pasted as a join key it errors “oauth authkeys require --advertise-tags” and creates Ephemeral nodes.
Turn Ephemeral OFF on the key.	Ephemeral nodes are deleted when they go offline (a reboot could remove your dev VM).
Tag the key with both `tag:devsys` and `tag:<dev>`.	The VM needs tag:devsys (admin reach) AND tag:<dev> (the dev SSH rule's target). A single-tagged key means hand-adding the other tag every time.
Add the per-dev SSH rule (`<dev>@…` → `tag:<dev>`, user `<dev>`).	“not permit you to SSH as user X” = the node matches but no rule grants that Unix user — add this rule.

Multi-user VM — one Claude login, two GitHub accounts

For a box where one person uses several Unix users (two GitHub identities: `soheb` = account A's repos, `nv` = account B's). Keep `gh` and `git` per-user (each logs into its own GitHub account) — that's the account split. Then `DEVSYS_SHARED_CLAUDE=1` lets all the VM's users share one Claude sign-in.

Concern	Scope	Result
Claude (auth, agents, settings, history)	shared: `CLAUDE_CONFIG_DIR=/opt/devsys/claude`	sign in once, all users use it
GitHub auth + git identity	per-user: `~/.config/gh`, `~/.gitconfig`	claude as soheb → acct A, as nv → acct B

Enable: bake into the base with `… | DEVSYS_UNATTENDED=1 DEVSYS_SHARED_CLAUDE=1 bash`, or retrofit a live VM: `sudo devsys update && sudo DEVSYS_SHARED_CLAUDE=1 DEVSYS_USER=<u> devsys install none`. Sign in once (`claude`); each user reads it after a fresh session.

New users are automatic: create them with `sudo DEVSYS_UNATTENDED=1 DEVSYS_USER=<name> devsys install none` — auto-enrolled in the `devs` group, reading the shared login with no sign-in (just a fresh login). Plain `adduser` does not — you'd `usermod -aG devs <name>` by hand.

Handled for you: Claude writes credential files mode 0600, which would re-lock other users on each token refresh; the flag installs a systemd path unit and a 1-minute timer (`devsys-claude-share`) that keep the shared dir group-readable. Scope is per-VM — never one Claude login across VMs. Skip the flag on single-developer VMs (pointless there).

Phase 4 — Verify & operate

Check / task	Command
Admin reach (rule 2)	`ssh root@sohebvm`
Dev login (rule 3)	`ssh soheb@sohebvm`
Isolation (must fail)	`ssh soheb@`<another dev's VM> → denied
Health / versions (in VM)	`devsys status` · `devsys --list`
Add a tool fleet-wide	edit `MISE_*`/`NPM_*` pins → rebuild base → re-clone
Bump the golden image	`incus start devsys-base` → re-install → `./build-base.sh` again
Break-glass into a VM	`incus exec <vm> -- bash` (on the host)
Retire a VM	delete in Incus + remove from Tailscale Machines

Validated on Incus (beehive): golden image built & published; clones boot with the pinned toolchain, join Tailscale dual-tagged & non-ephemeral, `ssh root@` (admin) and `ssh <dev>@` (dev) both connect, per-dev isolation holds. Shared-Claude (3.1.1) verified across root/soheb/nv on one VM with GitHub kept per-user.

Installers 3.2.0 and 3.3.0 (the group-skip fix, the shell setup and Mac locale fix, and the inbound firewall that 3.3.0 adds: Tailscale and loopback only, `DEVSYS_FIREWALL=0` skips it) were tested on OrbStack only — a full build and clones — and have not been run on Incus.